UT 8. Dockers

Sumario

1. Redes en Docker....2
   1.1. Tipos de redes en Docker....2
   1.2. Instalación de utilidades de red en un contenedor....3
   1.3. Gestionando redes....4
   1.4. Obteniendo información de las redes....7
   1.5. Asociando redes a contenedores....7
   1.6. Iptables en contenedores....8
   1.7. Bridge network driver....9
   a) Ejemplo Bridge por defecto:....12
   b) Bridge personalizado:....15
   1.8. HOST network driver....16
   1.9. Redes IPvlan....18
   a) Un ejemplo de IPvlan L2....18
   1.10. Redes macvlan....22
2. Aplicaciones multicapa con docker-compose....26
   2.1. Instalación de docker-compose....27
   2.2. El archivo docker-compose.yml....28
   a) Asociar contenedores a servicios....29
   b) Especificar las variables de entorno para los contenedores....29
   c) Establezco la redirección de puertos si fuera necesario....30
   d) Persisto los datos usando un bind mount para el servidor web....30
   e) Persistir los datos para que el servidor de base de datos use un volumen....31
   f) Definir redes para asociarlas a los contenedores....32
   g) Establecer orden de inicio de los contenedores....33
   h) Fichero completo de los pasos anteriores....33
   i) Ejemplos de dockers compose....35
   2.3. Puesta en marcha de los servicios....35
3. Gestión gráfica de docker e integración con otras aplicaciones....38
   3.1. Gestión de docker con el contenedor Portainer....38
   3.2. Docker en Visual Studio Code....39
4. Gestión gráfica de docker e integración con otras aplicaciones....39
   4.1. Gestión de docker con el contenedor Portainer....39
   4.2. Docker en Visual Studio Code....40

1. Redes en Docker

1.1. Tipos de redes en Docker

Cuando nuestro contenedor y los servicios que podemos tener instalados en él tienen algún tipo de conexión nosotros no hemos tenido que configurar nada y el contenedor ni sabe ni es consciente del funcionamiento de la red ni de la plataforma sobre la que funciona. El contenedor es red y sistema agnóstico.

Esto se consigue con un sistema de red en el que nosotros podemos "enchufar" distintos dispositivos de red a cada contenedor usando distintos drivers que pueden ser de los siguientes tipos:

- **Bridge:** Es el driver por defecto. Bridge es la red estándar y network driver por defecto, por lo que se crea una vez se inicia la plataforma de Docker y sus contenedores se conectan a ella, a menos que el usuario indique específicamente lo contrario. Usualmente se utilizan cuando las aplicaciones del cliente se ejecutan en contenedores independientes entre sí, pero que requieren de una comunicación.

  Este tipo de red se aplica a aquellos containers que se ejecutan en el mismo host del demonio de Docker. Cabe resaltar que cada uno de estos contenedores cuenta con su propia red, que es, además, independiente de la red del host donde se encuentran ubicados.

- **Host:** El contenedor usa directamente la red de mi máquina (el host). El controlador de redes en Docker denominado Host es el encargado de eliminar el aislamiento que pueda existir entre un contenedor de la plataforma y el host. De manera que, cuando la red se encuentra en modo host, tiene la posibilidad de utilizarse con el objetivo de mejorar el rendimiento. Al mismo tiempo, este controlador de redes docker funciona en los momentos donde un container requiera controlar una gran diversidad de puertos, debido a que no necesita realizar el proceso de traducción de direcciones de red y, además, no se crea un proxy de usuario para cada uno de los puertos.

  En este tipo de redes en Docker, el contenedor usará la misma dirección IP del servidor real que tenga el usuario. Este tipo de controlador de redes docker solo puede utilizarse en los hosts propios del sistema operativo Linux y no presenta compatibilidad con recursos como Docker Desktop versión MAC ni Windows, ni tampoco es compatible con Docker EE para los servidores del sistema operativo de Windows.

- **Overlay:** Un sistema que conecta distintos servicios docker de máquinas diferentes. Se utiliza para docker Swarm, que es la tecnología de docker para la orquestación de contenedores.

  Hace referencia a uno de los tipos de redes en Docker que cumple con la función de crear una red distribuida entre distintos hosts del daemon de Docker. Esto permite que se realice la conexión de diversos contenedores en diferentes nodos del sistema.

  Estas redes docker conecta varios de los daemon de Docker, lo que contribuye a la comunicación de los llamados servicios de enjambre o swarm services entre sí y con un contenedor de tipo independiente, o bien entre dos contenedores independientes que se encuentres en diferentes daemon de la plataforma. Estas labores permiten que se pueda eliminar la necesidad de llevar a cabo un enrutamiento a nivel de sistema operativo entre los contenedores de Docker.

- **Ipvlan.** Ipvlan indica una de las redes en Docker o red de contenedores que se encarga de ofrecerle a los usuarios una gestión y control total sobre las direcciones IPv4 e IPv6. El controlador Ipvlan también proporciona a los operadores la totalidad del manejo del etiquetado de tipo VLAN de capa 2, e incluso de elementos como el enrutamiento IPvlan L3 para aquellos usuarios que quieran realizar procesos de integración de la red subyacente.
- **MacVlan:** Que nos permite asignar una MAC a nuestro contenedor que parecerá que es un dispositivo físico en nuestra red.
- **None:** Si queremos que el contenedor no tenga conectividad alguna.

Aunque todos estos tipos de drivers tienen su utilizad en determinadas situaciones lo general es utilizar los drivers de tipo BRIDGE que me van a permitir, siempre dentro nuestra máquina local:

- Aislar los distintos contenedores que tengo en distintas subredes docker, de tal manera que desde cada una de las subredes solo podremos acceder a los equipos de esa misma subred.
- Aislar los contenedores del acceso exterior.
- Publicar servicios que tengamos en los contenedores mediante redirecciones que docker implementará con las pertinentes reglas de ip tables.

Un ejemplo de una configuración de una RED CONSTRUIDA CON EL DRIVER BRIDGE podría ser la siguiente:

1.2. Instalación de utilidades de red en un contenedor.

Instalar el comando ping y ip a un contenedor

Instalar Ping

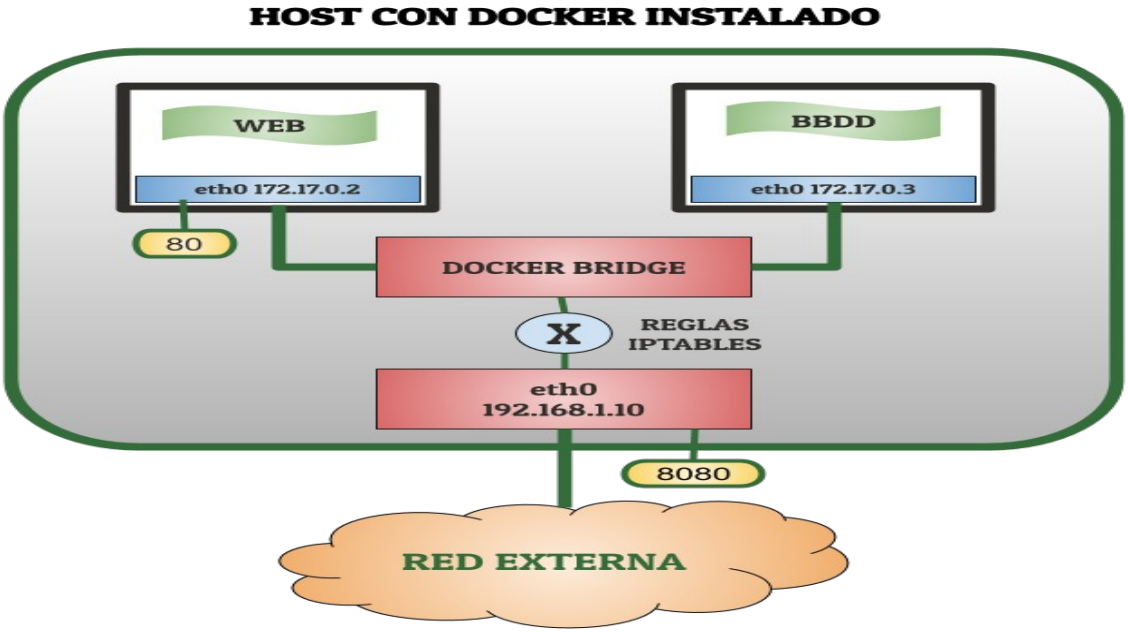
```
apt-get update -y
apt-get install -y iputils-ping
```

Instalar ip

```
apt-get update -y
apt-get install -y iproute2
```

Instalar dig

```
apt-get install dnsutils
```



1.3. Gestionando redes

Hay que hacer una diferenciación entre dos tipos de redes "bridged": la red creada por defecto por docker para que funcionen todos los contenedores y aquellas redes "bridged" definidas por el usuario, es decir, por nosotros. Esta red por defecto se llama bridge y podemos comprobrar que se ha creado ejecutando la siguiente orden que nos muestra todas las redes docker que tengamos:

Mostrar todas las redes docker creadas

```
docker network ls
```

Como resultado obtendremos una salida similar a la siguiente, en la que se destaca mediante un recuadro la red bridge por defecto.

```
NETWORK ID     NAME      DRIVER    SCOPE
303ec7f05904   bridge    bridge    local
25abd8ad8a91   host      host      local
fd1f9c0b0946   none      null      local
```

Esta salida, además del nombre de cada una de las redes creadas recoge la siguiente información:

- El NETWORK ID que me sirve para identificar una red y que se puede usar indistintamente con el nombre para cualquiera de las operaciones de gestión de redes (crear, borrar, obtener información etc...)
- El DRIVER, que como ya dijimos en el apartado anterior me define el tipo de red que voy a "conectar" a los contenedores. Podía tomar los valores bridge, none, host,macvlan y overlay.
- El SCOPE que nos indica el ámbito de nuestras redes y que en este caso son redes locales dentro de nuestra propia máquina.

Esta red "bridged" por defecto, que es la usada por defecto por los contenedores, se diferencia en varios aspectos de las redes "bridged" que creamos nosotros. Estos aspectos son los siguientes:

- Las redes que nosotros definamos proporcionan **resolución DNS entre los contenedores** cosa que la red por defecto no hace a no ser que usemos opciones que ya se consideran "deprectated".
- Puedo conectar en caliente a los contenedores redes "bridged" definidas por el usuario. Si uso la red por defecto tengo que parar previamente el contenedor.
- Me permite gestionar de manera más segura el aislamiento de los contenedores ,ya que si no indico una red al arrancar un contenedor éste se incluye en la red por defecto donde pueden convivir servicios que no tengan nada que ver.
- Tengo más control sobre la configuración de las redes si las defino yo. Los contenedores de la red por defecto comparten todos la misma configuración de red (MTU, reglas ip tables etc...).
- Los contenedores dentro de la red "bridge" comparten todos ciertas variables de entorno lo que puede provocar ciertos conflictos.

Una vez nos hemos situado vamos a ver cómo realizamos las operaciones más comunes para gestionar y trabajar con redes en docker. Estas operaciones son:

- Listado de las redes (ya visto, docker network ls)
- Creación de las redes. (docker network create)
- Inspeccionar redes (docker network inspect)
- Borrado de las redes. (docker network rm / docker network prune)

Una descripción más detallada de lo todas las opciones la podemos ver en la referencia completa de redes en docker pero, tal y como acostumbramos en este curso, vamos a ilustrar su funcionamiento con distintos ejemplos.

Ejemplos de creación de redes

Crear una red. Al no poner nada más coge las opciones por defecto, red bridge local y el mismo docker elige la dirección de red y la máscara

```
docker network create red1
```

Crear una red (la red2) dándole explícitamente el driver bridge (-d) , una dirección y una máscara de red (--subnet) y una gateway (--gateway)

```
docker network create -d bridge --subnet 172.24.0.0./16 --gateway 172.24.0.1 red2
```

La orden docker network create tiene más opciones para las redes de tipo bridge y muchas más para redes de otro tipo.

Nota: Cada red docker que creo crea un puente de red específico para cada red que podemos ver con ifconfig / ip a

Eliminación de redes

Eliminar la red red1

```
docker network rm red1
```

Eliminar una red con un determinado ID

```
docker network rm 3cb4100fe2dc
```

Eliminar todas la redes que no tengan contenedores asociados

```
docker network prune
```

Eliminar todas las redes que no tengas contenedores asociados sin preguntar confirmación (-f o --force)

```
docker network prune -f
```

Eliminar todas las redes que no tengan contenedores asociados y que fueron creadas hace más de 1 hora (--filter)

```
docker network prune  --filter until=60m
```

Nota: no puedo borrar una red que tenga contenedores que la estén usando. Deberé primero borrar los contenedores o desconectar la red.

1.4. Obteniendo información de las redes

De igual manera que con las imágenes y los contenedores, puedo obtener información de las redes docker de maneras diferentes:

- Mediante la orden **docker network ls**, que presentamos en el apartado anterior y que además tiene diversas opciones algunas de las cuales veremos posteriormente.
- Mediante la orden **docker network inspect**, que nos mostrará una información mucho más detallada con todas las características de la red.

1.5. Asociando redes a contenedores

En los tres apartados anteriores hemos hablado de aspectos relativos a las redes docker pero no hemos dicho nada de cómo "conectamos" esas redes a nuestros contenedores que es el paso fundamental para que dichos contenedores puedan conectarse entre ellos , puedan ofrecer servicios al exterior y puedan conectarse a Internet para poder actualizarse y/o instalar cualquier dependencia que necesitemos. Para realizar esta "conexión" debemos de tener en cuenta los siguientes aspectos:

- Al arrancar un contenedor podemos especificar a qué red está conectado inicialmente usando el flag --network seguido del nombre de la red a la que queremos conectarlo.
- Si al arrancar un contenedor no especificamos una red, el contenedor se conectará a la red por defecto, la red "bridge" que usa el driver "bridge".
- Al arrancar un contenedor no puedo "conectarlo" inicialmente a más de una red.
- Tras crear el contenedor puedo conectarlo a más redes o desconectarlo de alguna red. Dependiendo de si he elegido la red por defecto o no, podré o no podré hacer esa conexión o desconexión en caliente (con el contenedor funcionando).

Para ilustrar todas estar afirmaciones vamos a realizar distintos ejemplos:.

Arrancar un contenedor de Apache sin especificar red y habilitando la conexión desde el exterior a través del puerto 80. Se conectará por defecto a la red bridge.

```
docker run -d --name web -p 80:80 httpd
```

Arrancar un contenedor de Apache conectándose a la red red1 que es una red bridge definida por el usuario y habilitando la conexión desde el exterior a través del puerto 8080

```
docker run -d --name web2 --network red1 -p 8080:80 httpd
```

Arrancar un contenedor de Apache conectándose a la red red1 dándole una ip (que debe pertenecer a esa red)

```
docker run -d --name web2 --network red1 --ip 172.18.0.5 -p 8181:80 httpd
```

Conectar una nueva red, la red2 al contenedor web2.

```
docker network connect red2 web2
```

Conectar una red, la red2 al contenedor web y darle una ip (que debe pertenecer a esa red)

```
docker network connect --ip 172.28.0.3 red2 web
```

Desconectar la red1 del contenedor web2. Debe estar funcionando para poder desconectarse

```
docker network disconnect red1 web2
```

Por supuesto la orden docker connect tienen más opciones que podemos consultar en la referencia y , adicionalmente, hay varios flags de la orden docker run que están relacionados con redes y que pueden resultar de interés:

- --dns para establecer unos servidores DNS predeterminados.
- --ip6 para establecer la dirección de red ipv6
- --hostname o -h para establecer el nombre de host del contenedor. Si no lo establezco será el ID del mismo.

1.6. Iptables en contenedores

En ocasiones en algunos de los módulos (más relacionados con redes normalmente) puede que queramos montar ciertos entornos, compuestos por varias máquinas, en los que sea necesario que tengamos nosotros el control de las reglas iptables de las distintas máquinas que los conforman.

Normalmente esto se hace con distintas máquinas virtuales pero en vista de las posibilidades de red que nos dan los contenedores, ¿podría montar entornos de ese estilo con contenedores?. Por defecto esto no es posible en los contenedores, pero podemos conseguirlo de una de estas dos maneras:

- Arrancar el contenedor con el flag --cap-add=NET_ADMIN. Así le damos al contenedor la "capacidad" Linux de administrar la red.
- Arrancar el contenedor de con flag --privileged que le da todas las capacidades al contenedor que puede hacer lo mismo que se puede hacer desde el host. Esto, además de habilitar el uso de iptables, me permitiría cosas como "instalar docker dentro de un contenedor docker". Debemos de tener mucho cuidado con esta característica porque puede presentar varios problemas al levantar ciertas limitaciones a los cgroups..

Independientemente de la opción que elijamos, antes de poder usar iptables, y dado las pocas cosas que incluye por defecto un contenedor, deberemos:

- apt update (para recargar los repositorios que no vienen cargados por defecto).
- apt install -y iptables (para instalar la herramienta de firewall para Linux).

1.7. Bridge network driver

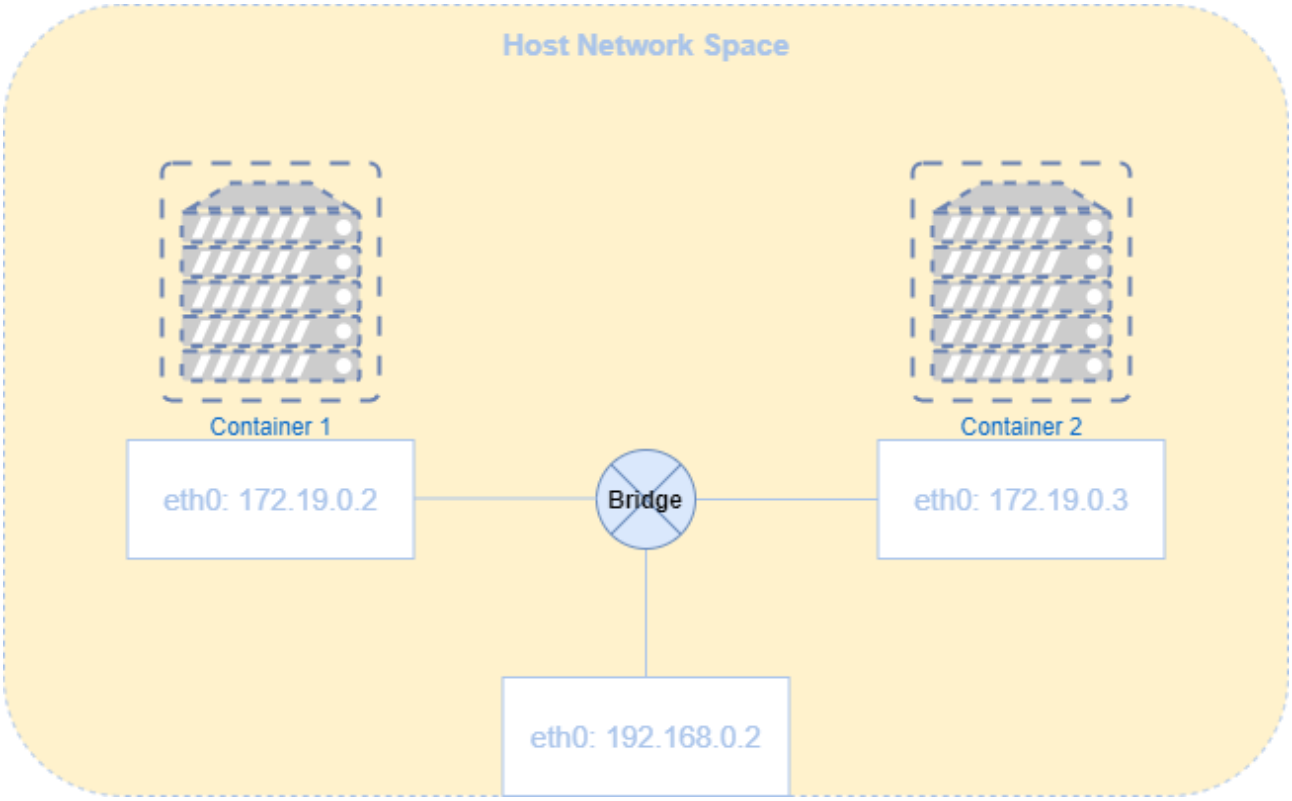
En términos de redes, una red puente es un dispositivo de capa de enlace que reenvía tráfico entre segmentos de red. Un puente puede ser un dispositivo de hardware o un dispositivo de software que se ejecuta dentro del kernel de una máquina host.

En términos de Docker, una red puente utiliza un puente de software que permite que los contenedores conectados a la misma red puente se comuniquen, al tiempo que proporciona aislamiento de los contenedores que no están conectados a esa red puente. El controlador del puente Docker instala automáticamente reglas en la máquina host para que los contenedores en diferentes redes puente no puedan comunicarse directamente entre sí.

Las redes puente se aplican a contenedores que se ejecutan en el mismo host demonio Docker. Para la comunicación entre contenedores que se ejecutan en diferentes hosts de

demonio Docker, puede administrar el enrutamiento a nivel del sistema operativo o puede usar una overlay network.

Cuando inicia Docker, se crea automáticamente una red puente predeterminada (también llamada bridge) y los contenedores recién iniciados se conectan a ella, a menos que se especifique lo contrario. bridgeTambién puede crear redes puente personalizadas definidas por el usuario. Las redes puente definidas por el usuario son superiores a la bridge red predeterminada.



Diferencias entre puentes definidos por el usuario y el puente predeterminado

- **Los puentes definidos por el usuario proporcionan resolución DNS automática entre contenedores .**

  Los contenedores en la red puente predeterminada solo pueden acceder entre sí mediante direcciones IP, a menos que use la –link opción , que se considera heredada. En una red puente definida por el usuario, los contenedores pueden resolverse entre sí por nombre o alias.

  Imagine una aplicación con un front-end web y un back-end de base de datos. Si llama a sus contenedores weby db, el contenedor web puede conectarse al contenedor de base de datos en db, sin importar en qué host Docker se esté ejecutando la pila de aplicaciones.

  Si ejecuta la misma pila de aplicaciones en la red puente predeterminada, deberá crear enlaces manualmente entre los contenedores (utilizando el --link indicador

heredado). Estos enlaces deben crearse en ambas direcciones, por lo que puede ver que esto se vuelve complejo con más de dos contenedores que necesitan comunicarse. Alternativamente, puede manipular los /etc/hosts archivos dentro de los contenedores, pero esto crea problemas que son difíciles de depurar.

- **Los puentes definidos por el usuario proporcionan un mejor aislamiento**

  Todos los contenedores sin un –network especificado se adjuntan a la red puente predeterminada. Esto puede ser un riesgo, ya que las pilas/servicios/contenedores no relacionados pueden comunicarse.

  El uso de una red definida por el usuario proporciona una red con alcance en la que solo los contenedores conectados a esa red pueden comunicarse.

- **Los contenedores se pueden conectar y desconectar de redes definidas por el usuario sobre la marcha .**

  Durante la vida útil de un contenedor, puede conectarlo o desconectarlo de redes definidas por el usuario sobre la marcha. Para eliminar un contenedor de la red puente predeterminada, debe detener el contenedor y volver a crearlo con diferentes opciones de red.

- **Cada red definida por el usuario crea un puente configurable .**

  Si sus contenedores usan la red puente predeterminada, puede configurarla, pero todos los contenedores usan la misma configuración, como MTU y iptables reglas. Además, la configuración de la red puente predeterminada se realiza fuera de Docker y requiere reiniciar Docker.

  Las redes puente definidas por el usuario se crean y configuran mediante docker network create. Si diferentes grupos de aplicaciones tienen diferentes requisitos de red, puede configurar cada puente definido por el usuario por separado, a medida que lo crea.

- **Los contenedores vinculados en la red puente predeterminada comparten variables de entorno .**

  Originalmente, la única forma de compartir variables de entorno entre dos contenedores era vincularlos usando la –link bandera . Este tipo de uso compartido de variables no es posible con redes definidas por el usuario. Sin embargo, existen formas superiores de compartir variables de entorno. Algunas ideas:

  - Varios contenedores pueden montar un archivo o directorio que contenga la información compartida, utilizando un volumen Docker.
  - Se pueden iniciar varios contenedores juntos docker-composey el archivo de redacción puede definir las variables compartidas.
  - Puede utilizar servicios de enjambre en lugar de contenedores independientes y aprovechar los secretos y configuraciones compartidos .

Los contenedores conectados a la misma red puente definida por el usuario exponen efectivamente todos los puertos entre sí. Para que un puerto sea accesible para contenedores o hosts que no sean Docker en diferentes redes, ese puerto debe publicarse usando el indicador -p o --publish .

a) Ejemplo Bridge por defecto:

Recordad que todo docker se conectarán por defecto bridge por defecto de docker que esta asociado al puente docker0

```
[root@localhost /]# ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
      valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
      valid_lft forever preferred_lft forever
---------- RESTO DE INTERFACES FÍSICAS,VIRTUAL BOX, VMWARE,.....
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
state DOWN group default
   link/ether 02:42:fa:21:6a:88 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
      valid_lft forever preferred_lft forever
```

Si inspeccionamos la red brigde por defecto son sale:

```
[root@localhost /]# sudo docker network inspect bridge
```

```
[
   {
      "Name": "bridge",
      "Id":
"dc61a8051f8e85fbd635f923b0ecd10968e0d4d15138125c436e14ccf86837bc",
      "Created": "2023-10-09T12:36:51.94158977+02:00",
      "Scope": "local",
      "Driver": "bridge",
      "EnableIPv6": false,
      "IPAM": {
         "Driver": "default",
         "Options": null,
         "Config": [
```

```
            {
                "Subnet": "172.17.0.0/16",
                "Gateway": "172.17.0.1"
            }
        ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
        "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {
        "com.docker.network.bridge.default_bridge": "true",
        "com.docker.network.bridge.enable_icc": "true",
        "com.docker.network.bridge.enable_ip_masquerade": "true",
        "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
        "com.docker.network.bridge.name": "docker0",
        "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {}
  }
]
```

Al crear un nuevo contenedor nuevo si no especificamos un red introduce este contenedor en la red bridge por defecto:

```
[root@localhost /]# docker run -itd --name=ubuntu ubuntu
```

Si lo inspecciono

el resultado es:

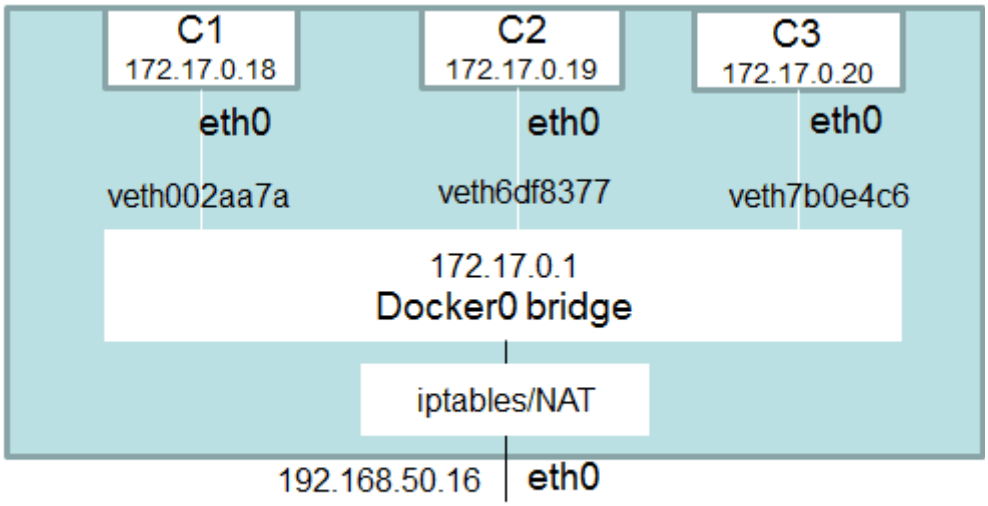
```
 "NetworkSettings": {
        "Bridge": "",
        "SandboxID":
"63bcef4ada1c51c1ca65906ea7e1554d3bbb31cd6c6e07654117c3c63584902a",
        "HairpinMode": false,
        "LinkLocalIPv6Address": "",
        "LinkLocalIPv6PrefixLen": 0,
        "Ports": {},
        "SandboxKey": "/var/run/docker/netns/63bcef4ada1c",
        "SecondaryIPAddresses": null,
        "SecondaryIPv6Addresses": null,
        "EndpointID":
"7be668eaadd54f9e0edee821efcf604e0667a21ff91f40ce93abe9421ca9b633",
        "Gateway": "172.17.0.1",
```

```
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "IPAddress": "172.17.0.2",
        "IPPrefixLen": 16,
        "IPv6Gateway": "",
        "MacAddress": "02:42:ac:11:00:02",
        "Networks": {
          "bridge": {
            "IPAMConfig": null,
            "Links": null,
            "Aliases": null,
            "NetworkID":
"542eff80d2d0e38a33519ce2da5b98805834f20e1f3cc6584766ff08c8c5bf6c",
            "EndpointID":
"7be668eaadd54f9e0edee821efcf604e0667a21ff91f40ce93abe9421ca9b633",
            "Gateway": "172.17.0.1",
            "IPAddress": "172.17.0.2",
            "IPPrefixLen": 16,
            "IPv6Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "MacAddress": "02:42:ac:11:00:02",
            "DriverOpts": null
          }
        }
    }
  }
```

Si hago ping a la puerta de enlace que es la ip del bridge por defecto

```
root@b1a27b3ff471:/# ping 172.17.0.1
PING 172.17.0.1 (172.17.0.1) 56(84) bytes of data.
64 bytes from 172.17.0.1: icmp_seq=1 ttl=64 time=0.102 ms
64 bytes from 172.17.0.1: icmp_seq=2 ttl=64 time=0.080 ms
```

Como podéis observar nuestro docker ya tiene una IP asignada automáticamente, si sucesivamente vamos generando dockers, tendremos todos colgando del interfaz docker0 y se verán entre ellos. Os dejo un esquema de cómo sería:



b) Bridge personalizado:

Vamos a crear dos redes bridge 10.10.0.0/24 y 10.20.0.0/24, y crear 4 containers, 2 a un red y 2 a la otra.

```
docker network create Bridge-01 --driver bridge --subnet 10.10.0.0/24 --gateway
10.10.0.1
docker network create Bridge-02 --driver bridge --subnet 10.20.0.0/24 --gateway
10.20.0.1
docker run -dit --name BG-01 --network Bridge-01  alpine ash
docker run -dit --name BG-02 --network Bridge-01  alpine ash
docker run -dit --name BG-03 --network Bridge-02  alpine ash
docker run -dit --name BG-04 --network Bridge-02  alpine ash
```

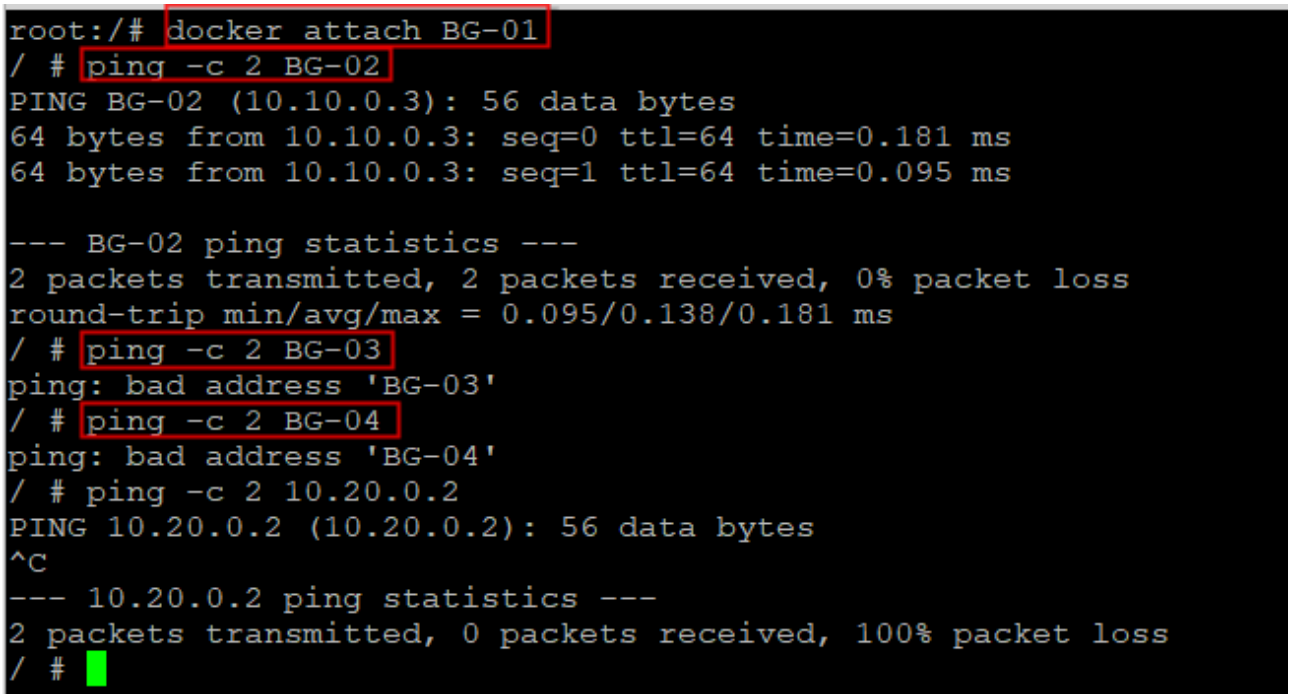
Me conecto al Container BG-01 (docker attach <Container>) que esta en la red Bridge-01, realizamos las siguientes pruebas:

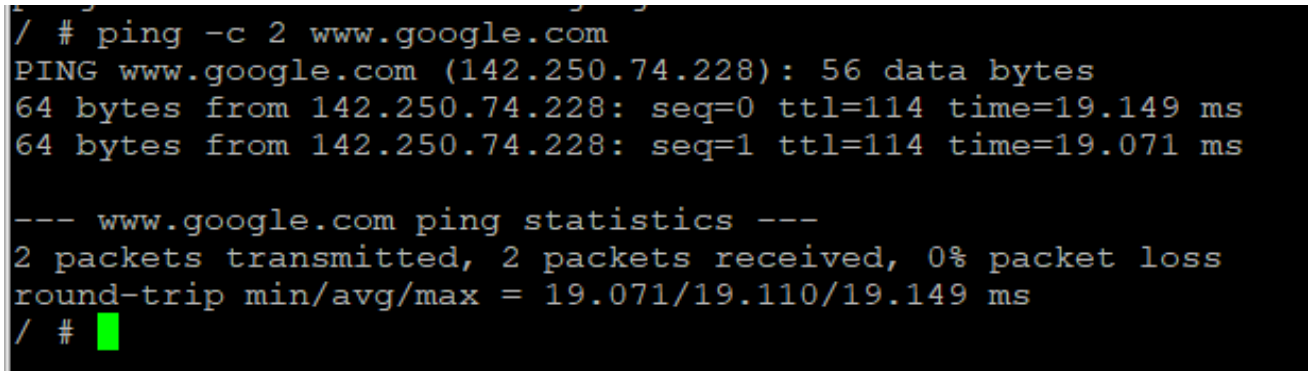
Ping al Container BG-02 en la Misma Red, responde OK.

Ping al Container BG-03 y BG-04 en la Distinta Red, responde KO.

Ping a IP BG-03 en Distinta Red, responde KO.

Ping a red externa OK.





1.8. HOST network driver

Las Host networks, son redes de un solo contenedor. Son especiales para los desarrolladores que llevan muchos proyectos y pueden necesitar activar y desactivar cada uno a la vez para retomar sus respectivas fases de desarrollo. No se requiere hacer mapeo de puertos, pues automáticamente todos los puertos del contenedor quedarán mapeados en el Docker Host. En pocas palabras: si levantamos un contenedor con NGINX, todos los puertos relacionados con el NGINX quedarán automáticamente reflejados en el Docker Host. Otro aspecto importante es que al utilizar este tipo de red, se perderá la capa de aislamiento, por lo tanto dependerá de una capa externa de seguridad para proteger al Docker Host y otorgar los accesos debidos a los puertos autorizados.

Docker ya viene con una red de tipo Host creada y no nos permitirá crear otra, pues no tiene sentido crear dos redes exactamente con las mismas limitaciones, si la que viene por defecto funciona perfectamente para lo que se necesita.

Para ponerla en funcionamiento, lo que haremos es, en principio, bajar todos los contenedores. La razón es sólo para evitar conflictos con el funcionamiento de puertos y de que puedan existir solapamientos en cuanto a los otros contenedores.

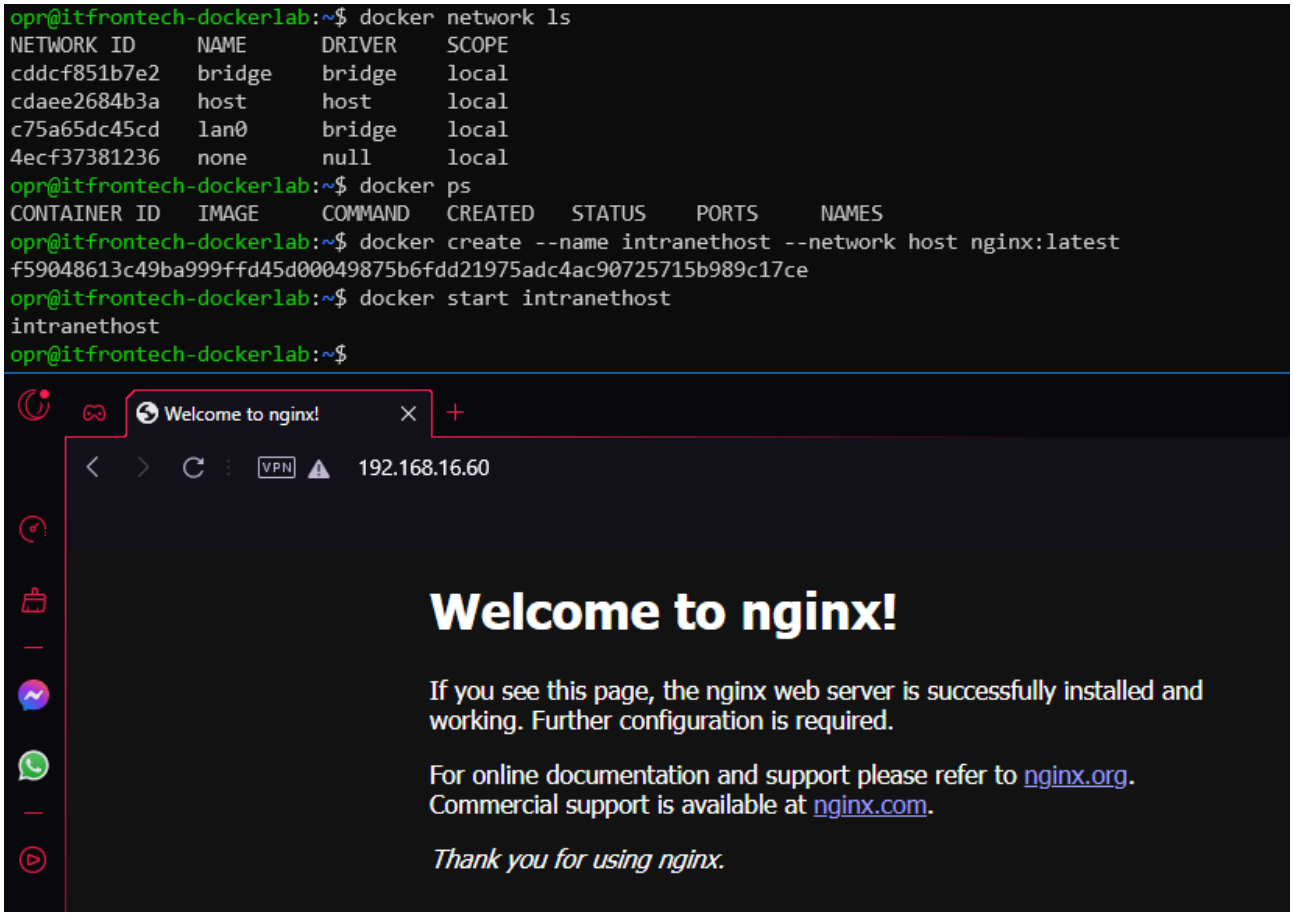
Ahora lo que haremos es crear el contenedor, al cual llamaremos intranethost y estará basado en una imagen de NGINX:

docker create --name intranethost --network host nginx:latest

Lo iniciamos:

```
docker start intranethost
```

Al entrar en el navegador e ir a la dirección IP del Docker Host veríamos a nuestro contenedor intranethost en acción :



No hay portmapping en lo absoluto. Este tipo de redes toman la red que tenga el Docker Host y sólo eso. Perderemos el aislamiento, pero podremos pasar de un proyecto a otro con la misma velocidad que ejecutemos un docker start nombre contenedor.

1.9. Redes IPvlan

Las redes IPvlan, o redes que hacen uso del controlador o driver IPvlan, son muy flexibles. Estas nos permiten crear un comportamiento similar al de las máquinas virtuales (VM), en cuanto a su forma de conectividad, sin embargo tiene ciertas ventajas que veremos posteriormente.

IPvlan nos da flexibilidad y autonomía sobre la configuración de las redes, tanto en IPv4 como en IPv6, y la gestión de los procesos de capa dos, concernientes al etiquetado de las redes virtuales (VLAN Tagging), como los de enrutamiento, concernientes a la capa tres. Estos procesos son llevados a cabo de una forma rápida y dinámica y se basan en la interface física de red (NIC), generando sub interfaces que permiten la separación lógica y el aislamiento entre redes a conveniencia de las necesidades y requerimientos del despliegue.

Lo anteriormente mencionado nos proporciona beneficios directos, tales como el evitar la utilización de los puentes (bridges) y que podamos trabajar directamente con la interface de red del Docker Host, mejorando el performance de la red de nuestros containers. Otro de los grandes beneficios es que no es necesario que utilicemos mapeo de puertos, permitiéndonos la implementación servicios que requieran la exposición de más de dos o más puertos en un mismo nodo / dirección IP.

a) Un ejemplo de IPvlan L2

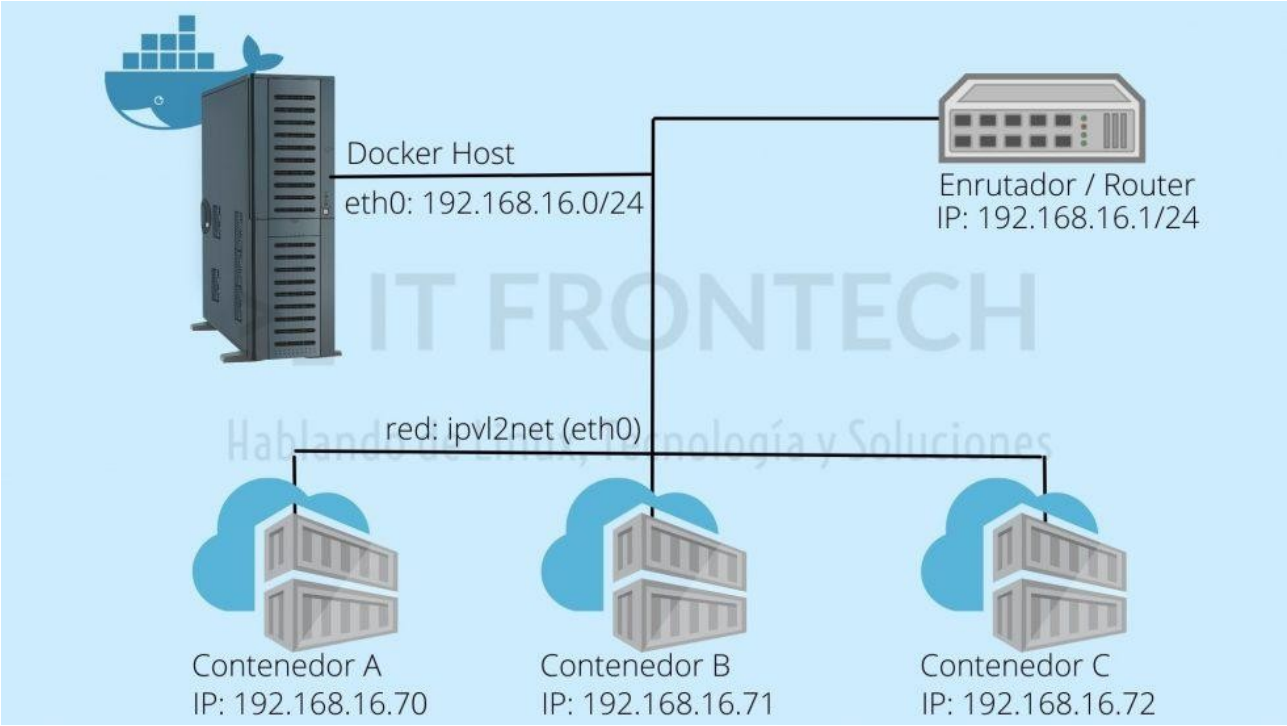
Partiendo del punto de que nuestra interface de red se llama eth0 y que nuestra subred en formato CIDR es la 192.168.16.0/24:

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:3b:be:af brd ff:ff:ff:ff:ff:ff
    inet 192.168.16.60/24 brd 192.168.16.255 scope global eth0
```

Imagen : Verificación de la red en el Docker Host

Implementaremos este ecosistema simple:

- Un (1) enrutador de red físico, el cual fungirá como puerta de enlace.
- Un (1) Docker Host
- Una (1) red de nombre ipvl2net, que utiliza el controlador tipo IPvlan, y que utiliza la interface eth0 del Docker Host y la misma subred física.
- Tres (3) contenedores, todos conectados a la red ipvl2net:
  - Contenedor_A: cuya dirección IP es la 192.168.16.70.
  - Contenedor_B: cuya dirección IP es la 192.168.16.71.
  - Contenedor_C: cuya dirección IP es la 192.168.16.72.



Para comenzar, lo primero que haremos es crear una red con controlador ipvlan, que llamaremos ipvl2net y cuya subred es la misma de la red física, por lo tanto también utilizará la misma puerta de enlace (la 192.168.16.1), para ello especificaremos que la interface padre de salida sea la eth0 y de este modo, tendrá exposición al a red física. Si no especificamos esto, entonces la red estará aislada.

```
docker network create -d ipvlan --subnet=192.168.16.0/24 --gateway=192.168.16.1 -o parent=eth0  ipvl2net
```

```
opr@itfrontech-dockerlab:~$ docker network create -d ipvlan --subnet=192.168.16.0/24 --gateway=192.168.16.1 -o parent=eth0 ipvl2net
68e60f58a96f82536b3315a6ef0ff18f9d0485f3ff23f0aaba5ecc0e04a024e8
opr@itfrontech-dockerlab:~$
```

Imagen : Creando la red ipvl2net con el controlador ipvlan

Ahora crearemos los contenedores. El primero será contenedor_A, con las siguientes características:

```
docker run --name contenedor_A --net=ipvl2net --ip=192.168.16.70 -dit --rm ubuntu
```

Si disectamos el comando veremos que:

- Utilizará la red que creamos anteriormente: ipvl2net (Con el parámetro –net=ipvl2net)
- Le asignaremos la dirección IP fija 192.168.16.70 (Con el parámetro –ip=192.168.16.70)
- Correrá en modo desatendido e interactivo (con los atributos -dit)

- Al terminar su tiempo de vida se eliminará automáticamente (Con los atributos –rm)
- Utilizará la imagen de Ubuntu

Y veremos de este modo como

```
opr@itfrontech-dockerlab:~$ docker run --name contenedor_A --net=ipvl2net --ip=192.168.16.70 -dit --rm ubuntu
5e94bec6f43d25edadcbd691ec023c33720cfef7f03ff374590ddfdd5392a38f
opr@itfrontech-dockerlab:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED              STATUS          PORTS     NAMES
5e94bec6f43d   ubuntu    "bash"    About a minute ago   Up 58 seconds             contenedor_A
opr@itfrontech-dockerlab:~$ _
```

Creando contenedor_A y conectándolo a la red ipvl2net

Imagen 5: Creando contenedor_A y conectándolo a la red ipvl2net

Ahora crearemos los dos contenedores faltantes y verificaremos, tal y como lo hicimos con el contenedor_A, que estos últimos estén corriendo:

```
opr@itfrontech-dockerlab:~$ docker run --name contenedor_B --net=ipvl2net --ip=192.168.16.71 -dit --rm ubuntu
b1977b9de7fbe3aefbbfa9f21c543044ae89e89580060c9b3265473b10775a1a
opr@itfrontech-dockerlab:~$ docker run --name contenedor_C --net=ipvl2net --ip=192.168.16.72 -dit --rm ubuntu
d80c4665b2831693a14bbe8dbdd0bed61c905a98394fc744b82c2709f8cea3e1
opr@itfrontech-dockerlab:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED          STATUS          PORTS     NAMES
d80c4665b283   ubuntu    "bash"    7 seconds ago    Up 6 seconds              contenedor_C
b1977b9de7fb   ubuntu    "bash"    21 seconds ago   Up 20 seconds             contenedor_B
5e94bec6f43d   ubuntu    "bash"    2 minutes ago    Up 2 minutes              contenedor_A
opr@itfrontech-dockerlab:~$ _
```

Imagen : Creación de los contenedores B y C y verificación de estado de funcionamiento

Inspeccionamos sus parámetros de red y verificamos que todo está correcto:

```
docker inspect contenedor_A contenedor_B contenedor_C | grep '"IPAddress":'
```

Y veremos las direcciones IP de cada uno de los contenedores consultados, de forma consecutiva. Todo está aparentemente bien.

```
opr@itfrontech-dockerlab:~$ docker inspect contenedor_A contenedor_B contenedor_C | grep '"IPAddress":'
            "IPAddress": "",
                    "IPAddress": "192.168.16.70",
            "IPAddress": "",
                    "IPAddress": "192.168.16.71",
            "IPAddress": "",
                    "IPAddress": "192.168.16.72",
```

Imagen : Revisión de las direcciones IP de los contenedores A, B y C con docker inspect

Solucionando el (posible) problema de redireccionamiento interno en Docker

Tenemos que editar el archivo o fichero /etc/sysctl.conf y editar la siguiente línea como root o super usuario:

```
sudo nano /etc/sysctl.conf
```

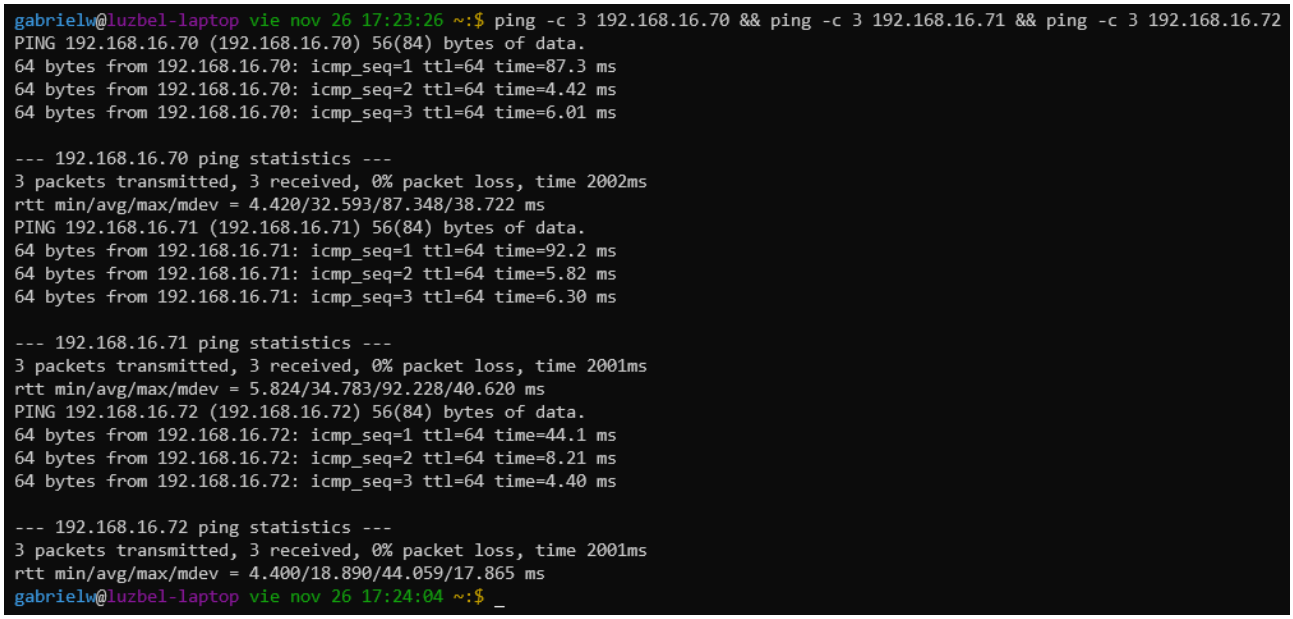
Y descomentaremos las líneas que dice net.ipv4.ip_forward y net.ipv6.conf.all.forwarding, estableciendo su valor a 1, sólo en caso de que no lo esté:

```
net.ipv4.ip_forward=1
net.ipv6.conf.all.forwarding=1
```

Guardar y salir del editor y luego ejecutar (como root):

```
sysctl -p
```

Y ahora hacemos ping desde otro nodo hacia nuestros contenedores:



Haciendo ping desde un nodo físico de la red hacia los contenedores ld1, ld2 y ld3

Hacer Ping del anfitrion a los contenedores:

Si desea tener conectividad de host a contenedor (y viceversa), debe crear manualmente una subinterfaz ipvlan en el espacio de nombres predeterminado en el host:

```
docker network create -d ipvlan --subnet=192.168.16.0/24 --gateway=192.168.16.1 -o
parent=eth0 \ ipvl2net
```

```
ip link add ipvl0 link eth0 type ipvlan mode l2
ip link set ipvl0 up
ip addr add 192.168.16.0/24 dev ipvl0
ip route add 192.168.16.0/24 dev ipvl0
```

Instalar el comando ping y ip a un contenedor

Instalar Ping

```
apt-get update -y
apt-get install -y iputils-ping
```

Instalar ip

```
apt-get update -y
apt-get install -y iproute2
```

1.10. Redes macvlan

Las redes macvlan son aquellas en las cuales creamos un dispositivo de red virtual con características de uno físico, es decir: al crearlas, tendremos la capacidad de asignarles una dirección física de red, como si se tratase de una NIC verdadera.

Las redes con driver macvlan conectarnos a puentes (bridges), vinculados con la tarjeta de red del Docker Host. Una de las formas más comunes de trabajar con dichas redes es crearlas y poder tener la capacidad de individualizar el tráfico

```
docker network create -d macvlan --subnet=192.168.16.0/24 --gateway=192.168.16.1
-o parent=eth0 macvln0
```

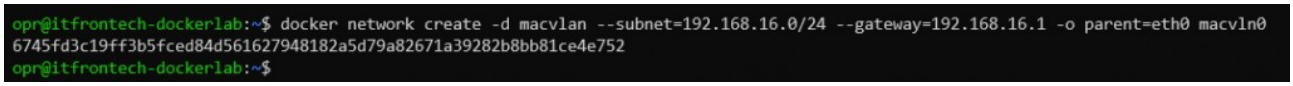


Imagen : Creando red con driver macvlan en Docker

NOTA: Una acotación importante es que si creamos este tipo de red con driver ipvlan o macvlan y estamos utilizando el mismo segmento de red de nuestra red física, NO declaremos el gateway, pues esto les causará problemas en su red, tanto en el Docker Host como en los contenedores, Puesto que al declarar la puerta de enlace, lo que hara Docker es crearla en el Docker Host con la misma dirección IP de nuestra puerta de enlace física y entonces colisionarán los paquetes. En ese caso, lo que haremos es crear la red macvlan de este modo:

```
docker network create -d macvlan --subnet=192.168.16.0/24 -o parent=eth0 macvln0
```

Bien, aquí ya hemos creado nuestra red y con sólo ejecutar un docker network ls podemos verla creada y el tipo de red que es:

```
opr@itfrontech-dockerlab:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
4e024f573730   bridge    bridge    local
16580cb5aa54   host      host      local
6745fd3c19ff   macvln0   macvlan   local
ea9bfa7a429b   none      null      local
opr@itfrontech-dockerlab:~$
```

Si tenemos una red, bien sea de tipo ipvlan o macvlan, y la creamos con la misma subred con la que creamos la red anterior, nos dará un error, por lo que deberemos eliminar la red anterior.

```
opr@itfrontech-dockerlab:~$ docker network create -d macvlan --subnet=192.168.16.0/24 --gateway=192.168.16.1 -o parent=eth0 macvln0
Error response from daemon: Pool overlaps with other one on this address space
```

Imagen : Error al crear una red macvlan en Docker

Bien, crearemos un contenedor con la red macvlan le asignaremos una MAC address :

```
docker run -itd --name macvlcontainer --net=macvln0 --ip=192.168.16.20 --mac-address=00:60:2F:8B:57:90 ubuntu:latest
```

```
opr@itfrontech-dockerlab:~$ docker run -itd --name macvlcontainer --net=macvln0 --ip=192.168.16.20 --mac-address=00:60:2F:8B:57:90 ubuntu:latest
e4c7a82101a467750fd23e21a17e046b7a9c4bb3973bc56a966628922bc27bc0
```

Imagen : Creando un contenedor con red macvlan e dirección IP fija

Ejecutamos docker ps -a para ver nuestros contenedores existentes. Recuerden que aun no lo hemos iniciado:

```
opr@itfrontech-dockerlab:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND    CREATED          STATUS                      PORTS     NAMES
e4c7a82101a4   ubuntu:latest  "bash"     7 seconds ago    Up 5 seconds                          macvlcontainer
ff59db8fa0b1   hello-world    "/hello"   2 weeks ago      Exited (0) 2 weeks ago                strange_poitras
opr@itfrontech-dockerlab:~$
```

Imagen 5: Listando contenedores

Ahora inspeccionaremos el contenedor que recién acabamos de crear y veremos la direcciones MAC e IP que le asignamos:



Imagen: Inspeccionando el contenedor macvlcontainer

Iniciamos el contenedor, accederemos a él y chequearemos si tiene conexión y podemos actualizar paquetes:

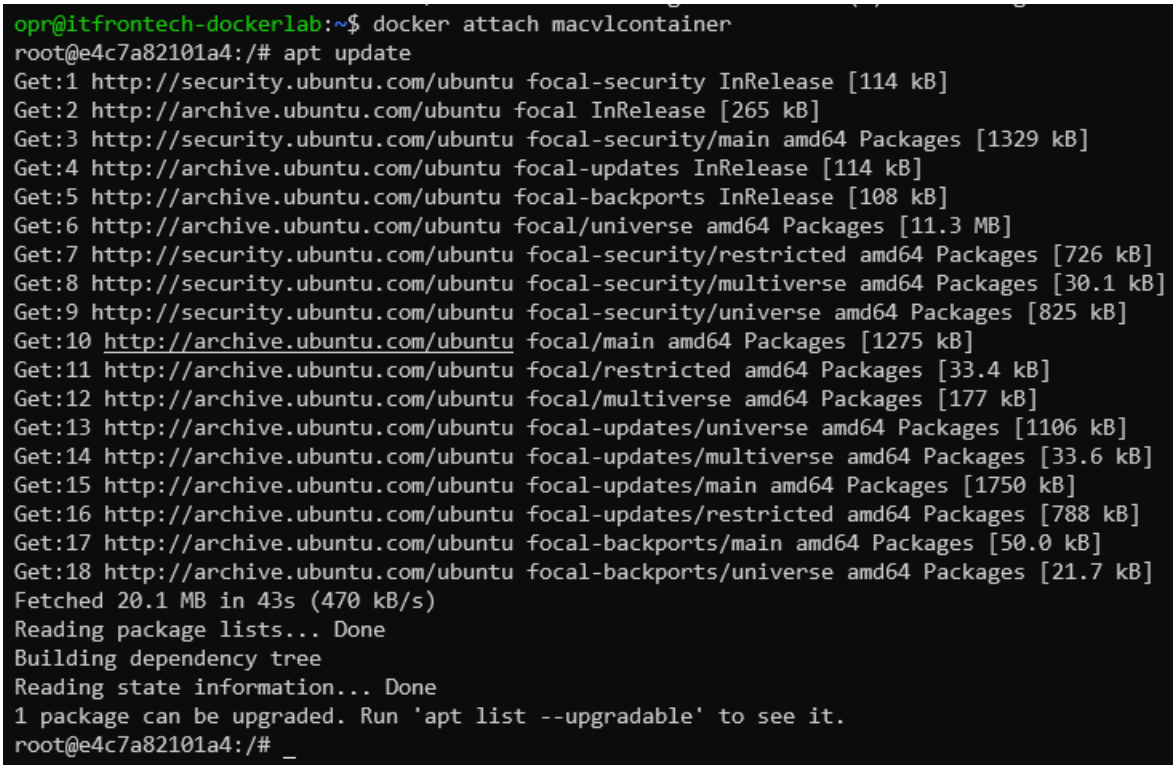
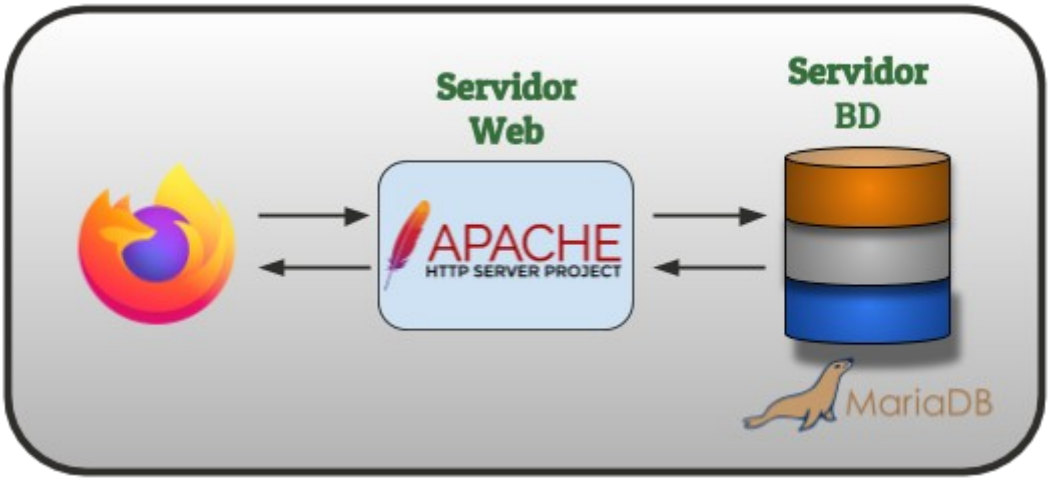


Imagen : Actualizando índices de repositorios en contenedor Docker

2. Aplicaciones multicapa con docker-compose

Hasta ahora hemos estado hablando de contenedores en solitario pero la realidad es que las aplicaciones actuales están formadas de varias aplicaciones o servicios. TÍPICAMENTE podríamos decir que tenemos al menos los siguientes elementos:



Arquitectura típica de una aplicación multi-capa

- Una CAPA DE PRESENTACIÓN que hace referencia al cliente que obtiene los datos pudiendo ser un navegador, una app móvil etc..
- Una CAPA LÓGICA que típicamente está representada por un servidor web, servidor de aplicaciones etc..
- Una CAPA DE DATOS que reside normalmente en un servidor de base de datos, ya sea relacional o no.

Además nos podemos encontrar aplicaciones con arquitecturas mucho más complejas con diversos servidores de bases de datos, distintos APIs independientes, servidores de autentificación etc...

Hasta ahora los contenedores los hemos creado y configurado por separado aunque conformaran una única aplicación. Tenía que arrancarlos y configurarlos uno a uno de manera manual. Eso no es ideal y además no es ágil, cada vez que quiero poner en funcionamiento este tipo de arquitectura deberé repetir todo el proceso de manera paso a paso. LO IDEAL sería:

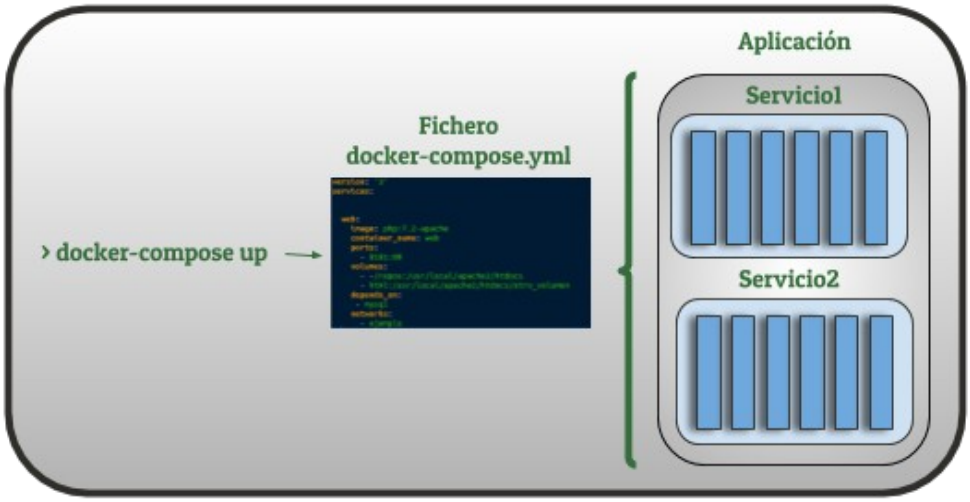
- Hacer todo de manera declarativa para que no tenga que repetir todo el proceso cada vez.
- Poner en funcionamiento todos los contenedores que necesita mi aplicación de una sola vez y debidamente configurados.

- Garantizar que los contenedores se arrancan en el orden adecuado. Por ejemplo: Mi aplicación no podrá funcionar debidamente hasta que no esté el servidor de bases de datos funcionando en marcha.
- Asegurarnos de que hay comunicación entre los contenedores que pertenecen a la aplicación.

Para todo esto tenemos la herramienta DOCKER-COMPOSE. Si tuviéramos que definir esta herramienta diríamos:

"Docker-compose es una herramienta para desplegar grupos de contenedores que forman parte de una misma aplicación o un mismo entorno"

El proceso para conseguir esto se describe de manera general en la siguiente imagen:



Los pasos son los siguientes:

1. Describo de manera declarativa todo los contenedores que conforman mi aplicación en el fichero docker-compose.yml. Este fichero es un un fichero con formato YAML.
2. Al ejecutar docker-compose up se levanta toda la aplicación, es decir, todos los contenedores que la conforman.

2.1. Instalación de docker-compose

La instalación de docker-compose es un proceso muy sencillo. Si somos usuarios de MAC y Windows no tendremos que instalar nada ya que docker-compose es una de las herramientas que por defecto se incluyen dentro de Docker Desktop.

Si somos usuarios de Linux su instalación se realiza únicamente con dos pasos:

1. Descarga del fichero mediante la orden curl y colocación en el directorio adecuado. Actualmente (Enero 2021) la versión vigente es la 1.27.4

```
sudo curl -L "https://github.com/docker/compose/releases/download/1.27.4/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

2. Concesión de los permisos de ejecución

```
sudo chmod +x /usr/local/bin/docker-compose
```

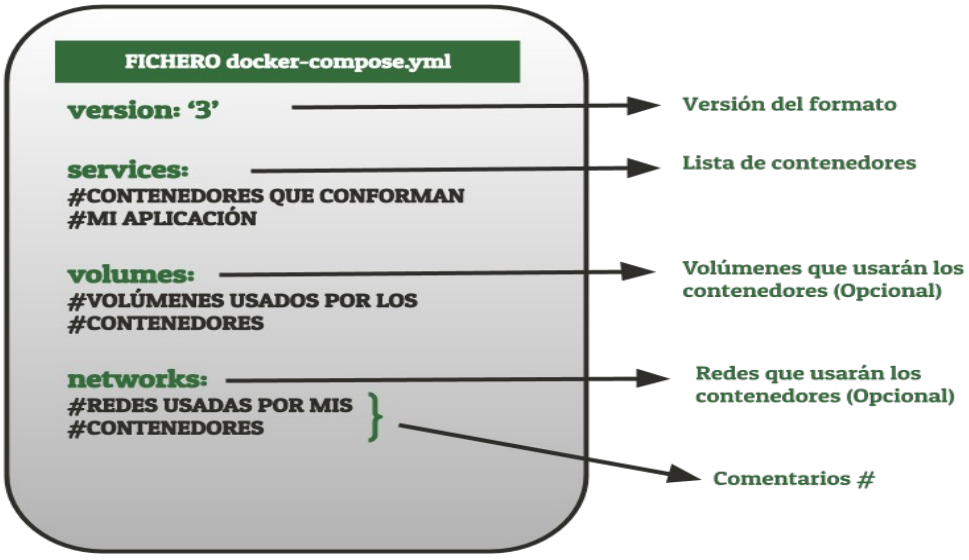
3. Comprobación de que la instalación está correcta.

```
docker-compose --version
```

```
docker-compose version 1.27.4, build 1110ad01
```

2.2. El archivo docker-compose.yml

Como ya hemos dicho en el apartado anterior el fichero **docker-compose.yml** es un fichero en formato YAML que contiene las instrucciones para crear y configurar los servicios que van a constituir mi aplicación o entorno. Su estructura general es la siguiente:



Dentro de cada una de estas secciones tenemos multitud de opciones. Es imposible tratar todas de manera detallada así que nos vamos a centrar en ejemplos para conseguir:

1. Asociar contenedores e imágenes a los servicios creados.
2. Especificar las distintas variables de entorno que pueden tener los contenedores creados. Recordad que hay que consultar siempre la página de cada imagen en DockerHub.
3. Establecer las redirecciones de puertos si fueran necesarias.
4. Persistir los datos de los contenedores usando bind mounts o volúmenes . Definiremos los volúmenes, si es necesario, para que sean usados por los contenedores.
5. Definir redes para asociarlas a los contenedores en caso de que queramos que no usen la red por defecto. En el módulo 5 hablamos de las diferencias entre la red por defecto y las redes creadas por el usuario.
6. Establecer orden de inicio para los contenedores que componen mi aplicación.

a) Asociar contenedores a servicios

```
version: '3'
 # Empieza la sección de servicios.
 Services:

  # Declaro un servicio con nombre miapache
  miapache:

   # Ese contenedor usará como imagen de base la imagen httpd(Servidor Apache) de DockerHub.
    image: httpd

   # Le doy nombre al contenedor cuando arranque (equivalente al flag --name de docker un)
    container_name: web
```

b) Especificar las variables de entorno para los contenedores

```
version: '3'

 # Empieza la sección de servicios.
 services:

  # Declaro un servicio con nombre datos
  datos:

   # Ese contenedor usará como imagen de base la imagen mariadb (Servidor de base de datos) de DockerHub.
    image: mariadb

    # Le doy nombre al contenedor cuando arranque (equivalente al flag --name de docker un)

    container_name: bd

    # Establezco las variables de entorno para configurar el servicio
    environment:
      MYSQL_ROOT_PASSWORD: 123456
      MYSQL_DATABASE: test
      MYSQL_USER: pepe

      MYSQL_PASSWORD: pepe
```

c) Establezco la redirección de puertos si fuera necesario

```
version: '3'

 # Empieza la sección de servicios.
 services:

  # Declaro un servicio con nombre miapache
  miapache:

   # Ese contenedor usará como imagen de base la imagen httpd(Servidor Apache) de DockerHub.
    image: httpd

    # Le doy nombre al contenedor cuando arranque (equivalente al flag --name de docker un)
    container_name: web

    # Establezco la redirección de puertos
    ports:

     - 8080:80
```

d) Persisto los datos usando un bind mount para el servidor web

```
version: '3'

 # Empieza la sección de servicios.
 services:

  # Declaro un servicio con nombre miapache
  miapache:

   # Ese contenedor usará como imagen de base la imagen httpd(Servidor Apache) de DockerHub.
    image: httpd

    # Le doy nombre al contenedor cuando arranque (equivalente al flag --name de docker un)
    container_name: web

    # Establezco un bind bound de la carpeta src de mi equipo en la carpeta /app del contenedor.
    volumes:
     - "./src:/app"

     # Notación alternativa a lo anterior
     - type: bind
       source: "./src"
       target: /app
```

e) Persistir los datos para que el servidor de base de datos use un volumen

```
version: '3'
 services:

  # Declaro un servicio con nombre datos
  datos:

   # Ese contenedor usará como imagen de base la imagen mariadb (Servidor de base de datos) de DockerHub.
    image: mariadb

    # Le doy nombre al contenedor cuando arranque (equivalente al flag --name de docker un)
    container_name: bd

    .........

    # Establezco que los datos de la base de datos van a persistir en el volumen datosapp que se montará en /var/lib/mysql

    volumes:
     - "datosapp:/var/lib/mysql"

    # Notación alternativa a lo anterior
```

```
    volumes:
      - type: volume
        src: datosapp
        target: "var/lib/mysql"

 # Sección para la definición de los volúmenes. Está al mismo nivel de la sección
services

  volumes:
    datosapp: local
```

f) Definir redes para asociarlas a los contenedores

```
version: '3'

# Empieza la sección de servicios.
  services:

    # Declaro un servicio con nombre miapache
    miapache:

     # Ese contenedor usará como imagen de base la imagen httpd(Servidor Apache)
de DockerHub.
      image: httpd

      # Le doy nombre al contenedor cuando arranque (equivalente al flag --name de
docker un)
      container_name: web
      # Establezco el nombre de red para el contenedor
      hostname: web

      # Establezco la red o redes a las que se va a conectar el contenedor

      networks:

        - ejemplo



# Sección de definición de redes. Está al mismo nivel que services y volumes

networks:

  # Definición de la red ejemplo
  ejemplo:
    # Tipo de red

    driver: bridge

    # Opciones de la red
    ipam:
      driver: default
```

```
    config:
      subnet: 172.20.0.0/16
```

g) Establecer orden de inicio de los contenedores

```
version: '3'
# Empieza la sección de servicios.

 services:
  # Declaro un servicio con nombre miapache

  miapache:
   # Ese contenedor usará como imagen de base la imagen httpd(Servidor Apache)
de DockerHub.

    image: httpd
    ....

   # Este contenedor arrancará después el contenedor del servicio datos
    depends_on:
      - datos

  # Comienzo de descripción del servicio de datos

   datos:

    image: mariadb
```

h) Fichero completo de los pasos anteriores

docker-compose.yml que une todo lo expuesto anteriormente.

```
version: '3'
services:

 #-------------------------------------------------------------------------
 # SERVICIO SERVIDOR WEB (php:7.4-apache con mysqli y código WP descargado
 #-------------------------------------------------------------------------
 web:
  # IMAGEN USADA
  image: jperjim398/miwp
  # NOMBRE QUE LE VOY A DAR AL CONTENEDOR
  container_name: web
  # REDIRECCIÓN DE PUERTOS
  ports:
   - 8181:80
  # SERVICIOS QUE TIENEN QUE ARRANCAR ANTES DE ARRANCAR ESTE
  depends_on:
   - datos
```

```
    # REDES A USAR
    networks:
      - ejemplo

  #------------------------------------------------
  # SERVICIO SERVIDOR DE BASE DE DATOS MARIADB
  #------------------------------------------------

  # NOMBRE DEL SERVICIO
  datos:
    # IMAGEN USADA
    image: mariadb
    # NOMBRE QUE LE VOY A DAR AL CONTENEDOR
    container_name: bd
    # LISTA DE VALORES DE ENTORNO CON SUS VALORES
    environment:
      MYSQL_ROOT_PASSWORD: 123456
      MYSQL_DATABASE: wordpress
      MYSQL_USER: pepe
      MYSQL_PASSWORD: pepe
    # REDIRECCIÓN DE PUERTOS
    ports:
      - 3316:3306
    # VOLÚMENES A USAR POR EL CONTENEDOR
    volumes:
      # DOCKER VOLUME
      - data:/var/lib/mysql
    # REDES A USAR
    networks:
      - ejemplo

# DEFINICIÓN DE VOLÚMENES DOCKER A USAR POR LOS SERVICIOS (OPCIONAL)
volumes:
  data:
    # TIPO DE DRIVER
    driver: local

# DEFINICIÓN DE LAS REDES A USAR POR LOS SERVICIOS (OPCIONAL)
networks:
  # NOMBRE DE LA RED
  ejemplo:
    # DRIVER DE LA RED
    driver: bridge
    ipam:
      driver: default
      config:
        - subnet: 172.20.0.0/16
```

Comprobación que funciona:

Acceder al navegador en la dirección http://localhost:8181

Realizar la configuración de Wordpress teniendo en cuenta que la configuración de la base de datos es:

- MYSQL_DATABASE: wordpress
- MYSQL_USER: pepe
- MYSQL_PASSWORD: pepe
- Servidor Mysql: 172.20.0.2:3306 (Mirar la ip que tiene asignada el contenedor db)

Podemos encontrar una referencia completa sobre el fichero docker-compose.yml en la documentación oficial: https://docs.docker.com/compose/compose-file/compose-file-v3/

i) Ejemplos de dockers compose

En internet podemos encontrar muchos ejemplos de docker-compose.

```
https://github.com/docker/awesome-compose/tree/master/wordpress-mysql
```

2.3. Puesta en marcha de los servicios

Una vez hemos creado el archivo docker-compose.yml tenemos que empezar a trabajar con él, es decir a crear los contenedores que describe su contenido.

Esto lo haremos mediante el ejecutable docker-compose.

Es importante destacar que debemos invocarla desde el directorio en el que se encuentra el fichero docker-compose.yml.

Esta herramienta (docker-compose) tiene muchos subcomandos y estos subcomandos, a su vez, tienen u montón de opciones. Nos vamos a centrar en ejemplificar con las combinaciones más comunes y además pondremos una lista de aquellas que también pueden resultar interesantes:

Obtener la versión de docker-compose.

```
docker-compose --version
```

Crear los contenedores (servicios) que están descritos en el docker-compose.yml.

```
docker-compose up
```

Crear en modo detach los contenedores (servicios) que están descritos en el docker-compose.yml.

Eso significa que no muestran mensajes de log en el terminal y que se nos vuelve a mostrar un prompt.

```
docker-compose up -d
```

Detiene los contenedores que previamente se han lanzado con docker-compose up.

```
docker-compose stop
```

Inicia los contenedores descritos en el docker-compose.yml que estén parados.

```
docker-compose run
```

Pausa los contenedores que previamente se han lanzado con docker-compose up.

```
docker-compose pause
```

Reanuda los contenedores que previamente se han pausado.

```
docker-compose unpause
```

Reinicia los contenedores. Orden ideal para reiniciar servicios con nuevas configuraciones.

```
docker-compose restart
```

Para los contenedores, los borra y también borra las redes que se han creado con docker-compose up (en caso de haberse creado).

```
docker-compose down
```

Para los contenedores y borra contenedores, redes y volúmenes

```
docker-compose down -v
```

Muestra los logs del servicio llamado servicio1 que estaba descrito en el docker-compose.yml.

```
docker-compose logs servicio1
```

Ejecuta una orden, en este caso /bin/bash en un contenedor llamado servicio1 que estaba descrito en el docker-compose.yml

```
docker-compose exec servicio1 /bin/bash
```

Algunos otros subcomandos interesante son:

- **docker-compose build** que ejecutaría, si está indicado, el proceso de construcción de una imagen que va a ser usado en el docker-compose.yml a partir de los ficheros Dockerfile que se indican.
- **docker-compose top** que muestra los procesos que están ejecutándose en cada uno de los contenedores de los servicios.

Importante (repitiendo): debemos llamar a la docker-compose desde el directorio donde está el fichero docker-compose.yml

Podéis encontrar la referencia completa de la orden docker-compose en el siguiente enlace: https://docs.docker.com/compose/reference/

Tarea. docker-compose para desarrollo

En esta práctica montaremos un entorno de desarrollo para el framework Ruby on Rails utilizando como base el docker-compose.yml de Bitnami que podemos encontrar en la siguiente URL: https://hub.docker.com/r/bitnami/rails/

1. Una vez hemos descargado el fichero docker-compose.yml asociado deberemos modificarlo de la siguiente manera:
   - Modificar el puerto de la aplicación al que nos podremos conectar desde nuestro host para que sea el puerto 8001. Recordad que sólo debemos modificar el primer puerto de la redirección.
2. Posteriormente deberemos iniciar la aplicación multicapa con docker-compose up.
3. Comprobar que está en ejecución a través de http://127.0.0.1:8001 en vuestro navegador.

Solución

```
sudo rm -R -f myapp/
```

```
mkdir ./myapp && cd ./myapp
chmod 777 myapp/
cd myapp/
 curl -LO https://raw.githubusercontent.com/bitnami/bitnami-docker-rails/master/
docker-compose.yml
nano docker-compose.yml
docker-compose up
```

Fichero docker-compose.yml

```
version: '2'
services:
  mariadb:
    image: docker.io/bitnami/mariadb:10.3
    environment:
      # ALLOW_EMPTY_PASSWORD is recommended only for development.
      - ALLOW_EMPTY_PASSWORD=yes

  myapp:
    tty: true # Enables debugging capabilities when attached to this container.
    image: docker.io/bitnami/rails:6
    environment:
      - DATABASE_HOST=mariadb
      - DATABASE_NAME=my_app_development
    depends_on:
      - mariadb
    ports:
      - 3000:3000
    volumes:
- .:/app
```

3. Gestión gráfica de docker e integración con otras aplicaciones

3.1. Gestión de docker con el contenedor Portainer

Se trata de una interfaz de usuario web que nos permite administrar fácilmente nuestro host Docker y todos los contenedores que en él habitan.

Portainer tiene como una de sus ventajas principales su facilidad de uso. Consiste en un solo contenedor que puede ejecutarse en cualquier motor de Docker, tanto en GNU/Linux como en otros sistemas, como Microsoft Windows.

Con esta herramienta podemos administrar las pilas de Docker, los contenedores, las imágenes, volúmenes y redes. Es compatible con Docker CE, Docker EE y Docker Swarm.

Despliegue de Portainer

Ya hemos dicho que el producto se encuentra dentro de un contenedor así que podemos desplegarlo igual que cualquier otro Docker.

```
docker volume create portainer_data

docker run -d -p 9000:9000 --name portainer --restart always \
-v /var/run/docker.sock:/var/run/docker.sock \
-v portainer_data:/data portainer/portainer
```

Lo que hace es escuchar en el puerto 9000. Allí es donde lo tendremos arrancado. Por tanto para acceder a él, simplemente debemos ir a **http://localhost:9000** en nuestro navegador. portainer/portainer, es el repositorio de DockerHub de donde descargamos la imagen de Portainer.

Os preguntará que pongáis un usuario y una contraseña

3.2. Docker en Visual Studio Code

Para instalar el complemento Docker en VS Code debemos ir a complementos, escribir Docker y en la primera opción de Docker-Microsoft .

4. Gestión gráfica de docker e integración con otras aplicaciones

4.1. Gestión de docker con el contenedor Portainer

Se trata de una interfaz de usuario web que nos permite administrar fácilmente nuestro host Docker y todos los contenedores que en él habitan.

Portainer tiene como una de sus ventajas principales su facilidad de uso. Consiste en un solo contenedor que puede ejecutarse en cualquier motor de Docker, tanto en GNU/Linux como en otros sistemas, como Microsoft Windows.

Con esta herramienta podemos administrar las pilas de Docker, los contenedores, las imágenes, volúmenes y redes. Es compatible con Docker CE, Docker EE y Docker Swarm.

Despliegue de Portainer

Ya hemos dicho que el producto se encuentra dentro de un contenedor así que podemos desplegarlo igual que cualquier otro Docker.

```
docker volume create portainer_data

docker run -d -p 9000:9000 --name portainer --restart always \
-v /var/run/docker.sock:/var/run/docker.sock \
-v portainer_data:/data portainer/portainer
```

Lo que hace es escuchar en el puerto 9000. Allí es donde lo tendremos arrancado. Por tanto para acceder a él, simplemente debemos ir a **http://localhost:9000** en nuestro navegador. portainer/portainer, es el repositorio de DockerHub de donde descargamos la imagen de Portainer.

Os preguntará que pongáis un usuario y una contraseña

4.2. Docker en Visual Studio Code

Para instalar el complemento Docker en VS Code debemos ir a complementos, escribir Docker y en la primera opción de Docker-Microsoft .